

Networking for Connected Devices: TCP/IP, MQTT, and TLS

The Client and Server Model, Long-Lived Connections, and Why an ESP32 Needs the Correct Time Before It Can Trust a Certificate

Report Title	Networking for Connected Devices: TCP/IP, MQTT, and TLS
Program	IoT Training Program
Module	Day 17: TCP/IP Client and Server Model, and TLS
Date	10 / 8 / 2026
Prepared By	Malak Alsharqawi
Organization	Cyber Robot

Contents

Contents

Contents	2
Section 1: The TCP/IP Client and Server Model	4
1.1 Two Ways to Talk: The Letter and the Phone Call	4
1.2 Where TCP Sits in the Stack	4
1.3 What TCP Is Responsible For	5
1.4 The Two Roles: Who Listens and Who Initiates	6
1.5 IP Addresses and Ports	6
1.6 Opening the Connection: The Three-Way Handshake	7
Step 1: SYN	7
Step 2: SYN and ACK	7
Step 3: ACK	8
1.7 One Connection, Many Messages	8
1.8 Why Not Reconnect for Every Message?	8
1.9 TCP Is Full Duplex	9
1.10 Keeping a Long-Lived Connection Alive	9
1.11 TCP Carries Bytes, Not Messages	10
1.12 How This Differs from HTTP Request and Response	11
1.12.1 The server cannot start a conversation	11
1.12.2 Every request carries its own context	11
1.12.3 HTTP does not necessarily open a new connection each time	11
1.13 How This Differs from MQTT and Its Broker	11
1.14 The Three Models Side by Side	12
1.15 How Much Does the Wrapper Actually Cost?	13
1.16 Choosing Between Them	13
1.17 Applying This to the ESP32 Work	14
1.18 Section 1 Recap	14
Section 2: TLS, What It Actually Adds	16
2.1 Where TLS Fits	16
2.2 What Plain TCP Leaves Exposed	16
2.2.1 Anyone on the path can read it	16
2.2.2 Anyone can pretend to be the broker	17
2.2.3 Anyone in the middle can change it	17
2.3 The Three Guarantees TLS Provides	17
2.4 The TLS Handshake	17
2.4.1 ClientHello	18
2.4.2 ServerHello	18
2.4.3 Certificate	18

2.4.4 CertificateVerify	18
2.4.5 Finished	18
2.5 Two Handshakes, Not One	19
2.6 What a Certificate Actually Is	19
2.7 What the Device Checks	20
2.8 TLS Does Not Change Your Data Format	20
2.9 What an Observer Can Still See	20
2.10 Why the Clock Matters: notBefore and notAfter	21
2.11 The Two Failure Modes	21
2.11.1 The clock is too early	21
2.11.2 The clock is too late	21
2.12 Why This Hits Embedded Devices Specifically	22
2.13 NTP and SNTP	22
2.14 SNTP on the ESP32 in Practice	22
2.15 The Correct Startup Order	23
2.16 The Whole Picture, End to End	24
2.17 A Debugging Checklist	25
2.18 Common Misconceptions	26
2.19 Section 2 Recap	26
Final Takeaway	27
References	28

Section 1: The TCP/IP Client and Server Model

Every connected device eventually faces the same question: once two machines can reach each other over a network, how do they actually hold a conversation? An ESP32 on a workbench and a broker running on a laptop are both just numbers on a network until something defines who speaks first, how long they stay in touch, and what the bytes crossing between them are supposed to mean.

TCP is the answer to the first part of that question, and it is the layer almost everything else in this project quietly depends on. MQTT uses it. HTTP uses it. A custom protocol written from scratch would use it too. Beginners often meet MQTT first and never look underneath, which leads to a set of confusions that are difficult to shake off later: the belief that MQTT competes with TCP, that HTTP always opens a fresh connection, or that a socket is a kind of message. This section takes the layer underneath seriously, because everything in Section 2 sits on top of it.

THE SINGLE SENTENCE TO REMEMBER

TCP is a pipe, not an envelope. It guarantees that the bytes you push in at one end come out at the other end, complete and in order. It has no opinion whatsoever about what those bytes mean.

1.1 Two Ways to Talk: The Letter and the Phone Call

Before any of the technical detail, it helps to have a picture of the two conversation styles that keep coming up.

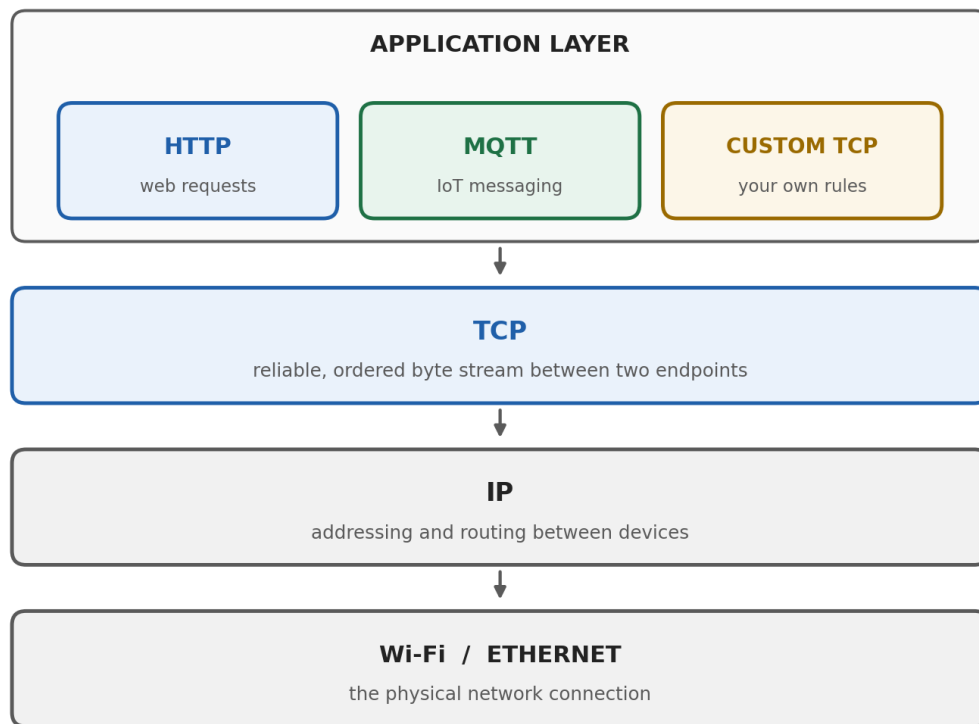
The first style is the letter. You write it, address it, post it, and it arrives on its own. Every letter is independent. If you want to say five things, you post five letters, and each one has to carry the full address and the full context all over again. Nothing connects them except the fact that you wrote them all.

The second style is the phone call. You dial once. Someone picks up. From that moment on, both of you can speak whenever you like, in either direction, without redialling and without repeating who you are. The line stays open until one of you hangs up.

A long-lived TCP connection is the phone call. Understanding that single image removes most of the confusion in this section, because almost every practical difference between TCP, HTTP, and MQTT comes down to how each one treats that open line.

1.2 Where TCP Sits in the Stack

Networking is built in layers, and each layer only has to solve its own problem and trust the layer below it. A simplified version of the stack running inside an ESP32 project looks like this.



HTTP and MQTT are not alternatives to TCP. They sit on top of it.

Figure 1. The layers inside a connected embedded project. Each layer solves one problem and hands the rest downward.

Reading that diagram from the bottom upward:

- Wi-Fi or Ethernet moves electrical or radio signals between the device and the access point. It has no idea what the data is for.
- IP gives every device an address and works out how packets travel from one address to another, possibly across many networks. IP is best effort, which means it does not promise that anything arrives.
- TCP takes that unreliable delivery service and builds a reliable, ordered, two-way byte stream on top of it. This is where retransmission, ordering, and flow control happen.
- The application layer is where HTTP, MQTT, or a protocol you invent yourself decides what the bytes actually mean.

The important consequence, and the one worth repeating until it becomes obvious, is that HTTP and MQTT are not alternatives to TCP. They are passengers riding on it. Saying that MQTT is faster than TCP is a little like saying that a delivery company is faster than the motorway it drives on.

1.3 What TCP Is Responsible For

TCP, specified today in RFC 9293, which brought together decades of amendments to the original 1981 specification, is responsible for a specific and quite long list of jobs:

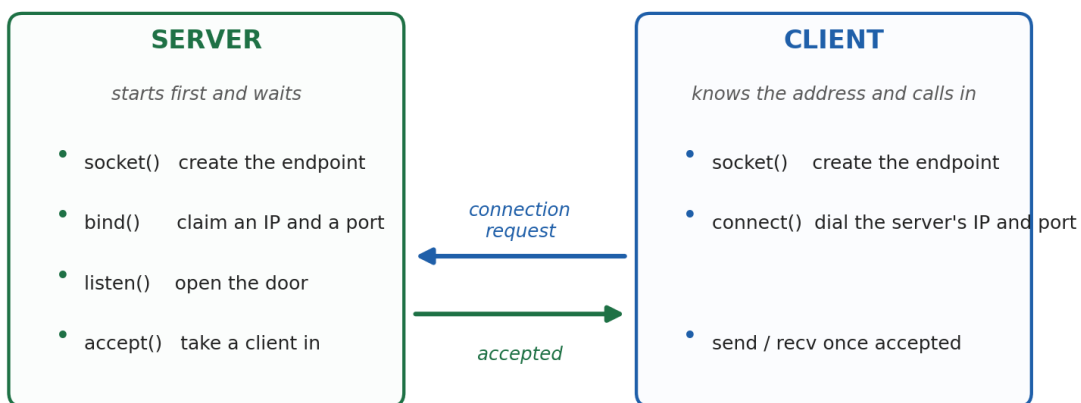
- Establishing a connection between two endpoints before any data is sent.
- Delivering data reliably, by retransmitting anything that is lost.
- Delivering data in the same order it was sent, even when packets arrive out of order.
- Detecting corruption using checksums.
- Controlling the rate of sending so a fast sender does not overwhelm a slow receiver, which is flow control.

- Backing off when the network itself is congested, which is congestion control.
- Closing the connection cleanly when either side is finished.

It is just as useful to be clear about what TCP does not do. TCP does not encrypt anything. TCP does not authenticate anyone. TCP does not know where one message ends and the next begins. TCP does not route messages to multiple recipients. Every one of those gaps is filled by a layer above it, and two of them are the entire subject of Section 2.

1.4 The Two Roles: Who Listens and Who Initiates

A TCP connection is always between exactly two endpoints, and those two endpoints do not play the same part in setting it up. One of them waits, and the other one calls.



The server never dials out. It only answers. The client always makes the first move.

Figure 2. The server prepares and waits. The client knows the address and initiates. The roles are only about who starts the connection, not about who is more important.

Role	What it does	In this project
Server	Starts first, claims an IP address and port, and waits for incoming connections. It never dials out.	The MQTT broker, or a PC running a TCP listener
Client	Knows where the server is and makes the first move by opening the connection.	The ESP32, in essentially every scenario
Both	Once the connection is accepted, both sides can send and receive freely.	Sensor data going up, commands coming down

In socket terms the server calls `socket`, then `bind`, then `listen`, then `accept`. The client calls `socket` and then `connect`. Those two short sequences are the whole of the client and server model.

One point catches beginners out constantly: client and server describe who opened the connection, not who sends more data or who matters more. An ESP32 that publishes a reading every second and receives one command an hour is still the client, because it was the one that dialled.

1.5 IP Addresses and Ports

An address alone is not enough to reach a program. The IP address identifies the device on the network, and the port number identifies which application on that device should receive the data. Written together they look like this:

```
192.168.1.100:5000
```

The first part is the device. The second part is the service running on it. A single device can therefore run many services at once without any confusion, because each one is listening on a different port.

Port	Usual service	Note
80	HTTP	Plain, unencrypted web traffic
443	HTTPS	HTTP protected by TLS
1883	MQTT	Plain MQTT, no encryption
8883	MQTT over TLS	The secure variant used in Section 2
123	NTP and SNTP	Time synchronisation, and it runs over UDP rather than TCP
5000	Anything you like	A common choice for a custom TCP application

Notice the last row of that table. Ports are conventions, not laws. What makes port 1883 an MQTT port is that an MQTT broker chose to listen there and MQTT clients expect to find it there.

1.6 Opening the Connection: The Three-Way Handshake

Before a single byte of application data moves, TCP performs a short negotiation called the three-way handshake. Its purpose is to confirm that both sides exist, that both are willing to talk, and that both agree on the sequence numbers they will use to keep the data in order.

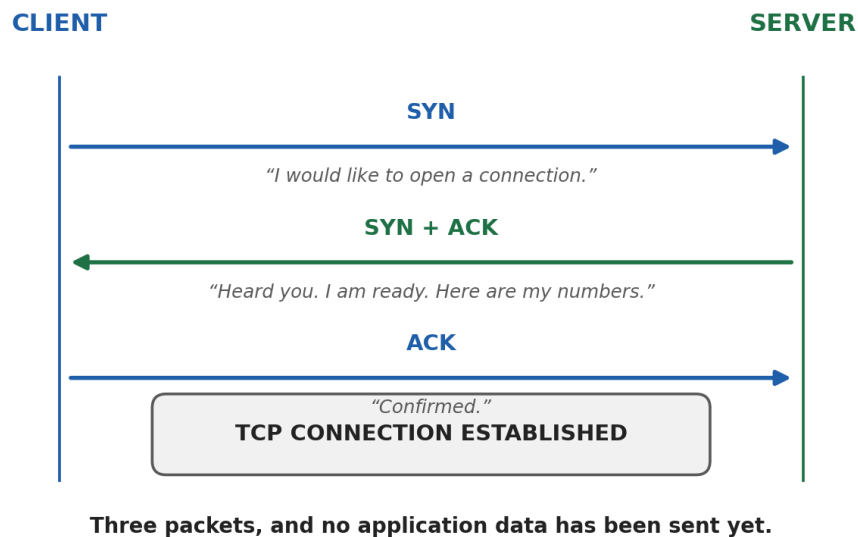


Figure 3. The three packets that open a TCP connection. No application data has been sent at this point.

Step 1: SYN

The client sends a packet with the SYN flag set, which is short for synchronise. In plain language it says: I would like to open a connection, and here is the sequence number I intend to start counting from.

Step 2: SYN and ACK

The server replies with a packet that both acknowledges the client's request and carries its own SYN. In plain language: I heard you, I am willing, and here is the sequence number I will start from.

Step 3: ACK

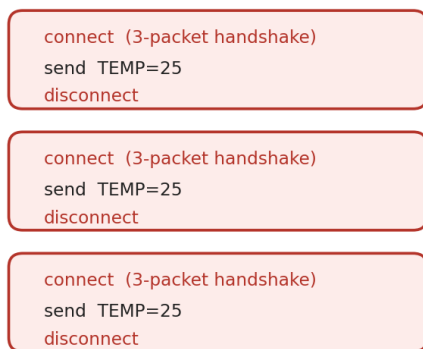
The client acknowledges the server's sequence number. The connection is now established, and application data can begin to flow. The client can in fact attach its first data to this third packet, so in practice the cost of opening a connection is one round trip before useful data moves.

That cost sounds small, and on a desk with a good access point it is. Over a congested Wi-Fi network, a mobile link, or a connection to a broker in another country, a round trip can be tens or hundreds of milliseconds. Paying it once is fine. Paying it every second is the subject of the next few pages.

1.7 One Connection, Many Messages

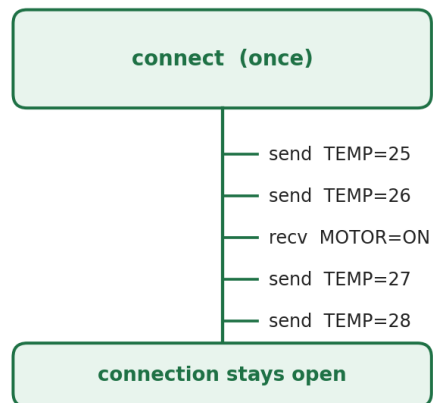
Here is the concept that most beginners have to unlearn something in order to accept. A TCP connection is not a message. It is a channel that stays open, and any number of messages can travel through it in both directions before it is closed.

RECONNECT FOR EVERY MESSAGE



setup cost paid again every single second

ONE LONG-LIVED CONNECTION



setup cost paid once, then pure data

Figure 4. The same five readings, sent two ways. On the left, the connection cost is paid five times. On the right, it is paid once.

A connection that is deliberately kept open across many exchanges is called a long-lived or persistent connection. There is no special TCP feature that creates one. A long-lived connection is simply a connection that neither side has closed. The application decides how long that is: a few seconds, the length of a session, or the entire uptime of the device.

1.8 Why Not Reconnect for Every Message?

Suppose an ESP32 reports temperature once per second. It could connect, send, and disconnect each time. It would work. It is also wasteful in several ways at once, and the waste is worth spelling out because none of it is visible in the code.

- Setup cost. Every connection repeats the three-way handshake, which is a full round trip before the reading can even be sent.

- Security cost. If the connection is protected, a full TLS handshake has to run as well, which adds at least one more round trip and a certificate verification each time. This is the single biggest cost in a secured system.
- Application cost. An MQTT client that reconnects must send CONNECT, wait for CONNACK, and resubscribe to its topics before it is useful again.
- Server cost. Each connection consumes a socket, memory, and often an entry in a connection table. Ten devices reconnecting every second is a very different load from ten devices connected once.
- Power cost. On a battery device, keeping the radio awake for repeated handshakes is usually more expensive than holding one idle connection.
- Latency cost. A command travelling down to the device can only arrive while a connection exists. A device that connects for fifty milliseconds a second is unreachable for the other nine hundred and fifty.

A WORKED COMPARISON

One reading per second for one hour, reconnecting each time: 3,600 TCP handshakes, 3,600 TLS handshakes if the link is secured, 3,600 MQTT CONNECT exchanges, and 3,600 teardowns. That is roughly fourteen thousand extra round trips to carry 3,600 short readings.

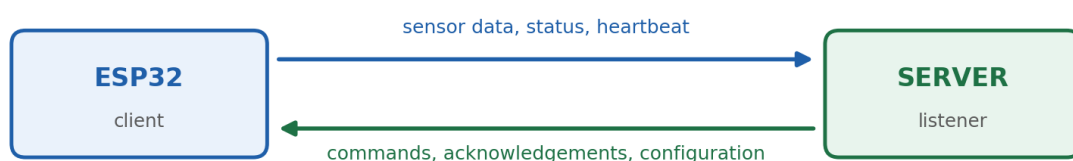
The same hour on a persistent connection: one TCP handshake, one TLS handshake, one MQTT CONNECT, then 3,600 small publishes and a handful of keep alive packets.

The payload is identical in both cases. Everything else is overhead you chose to pay.

There are situations where reconnecting is the right answer. A device that wakes from deep sleep once an hour, sends one reading, and sleeps again has nothing to keep alive, and holding a connection open through deep sleep is not possible in the ordinary case anyway. The rule is not that persistent connections are always better. The rule is that frequent communication belongs on a persistent connection, and rare communication does not need one.

1.9 TCP Is Full Duplex

Once the handshake is complete, the distinction between client and server largely stops mattering for data. Both sides can send at any time, and neither has to wait for the other to finish.



Once the connection exists, either side may speak at any time. Neither waits for permission.

Figure 5. Data flows in both directions independently. The server does not need an invitation to send a command.

This is more useful than it first appears, and it is exactly what a request and response protocol cannot easily do. If a server holding an open TCP connection decides at three in the morning that the motor must stop, it can simply send the command. There is no need for the device to ask first. Live sensor systems, control systems, chat applications, streaming, and industrial links all depend on this property.

1.10 Keeping a Long-Lived Connection Alive

A connection that is meant to last for days introduces a problem that a short connection never has: how does either side know the other is still there? If a device loses power or Wi-Fi drops, no packet is sent to announce it. The server may sit with a socket it believes is healthy, talking to nobody. This is called a half-open connection, and it is one of the classic sources of ghost devices in an IoT dashboard.

The practical answers are:

- Application level keep alive. MQTT includes a keep alive interval agreed at connect time. If the broker hears nothing within roughly one and a half times that interval, it considers the client gone. The client sends a small PINGREQ packet when it has nothing else to say.
- Last will and testament. An MQTT client can register a message in advance that the broker publishes on its behalf if the connection dies unexpectedly. It is a clean way to mark a device offline without polling it.
- TCP keep alive. The transport layer has its own optional mechanism, but its default timers are measured in hours on many systems, so application level keep alive is usually the practical choice.
- Reconnect with backoff. When the connection does drop, retrying immediately and forever is a good way to hammer a broker. Retrying after one second, then two, then four, up to a sensible ceiling, is kinder and more reliable.
- Network address translation timeouts. Home routers quietly drop idle connection entries after a few minutes. A keep alive interval shorter than that timeout is often what keeps a link alive across a router.

1.11 TCP Carries Bytes, Not Messages

This is the most commonly missed property of TCP, and it causes bugs that look like data corruption but are not.

TCP is a byte stream. It guarantees that every byte arrives, once, in order. It does not guarantee that the receiver's read calls line up with the sender's write calls. Three separate sends can arrive as one read, or one send can arrive split across two reads. TCP is free to combine or divide the data as it sees fit, because from its point of view there is only one continuous stream.

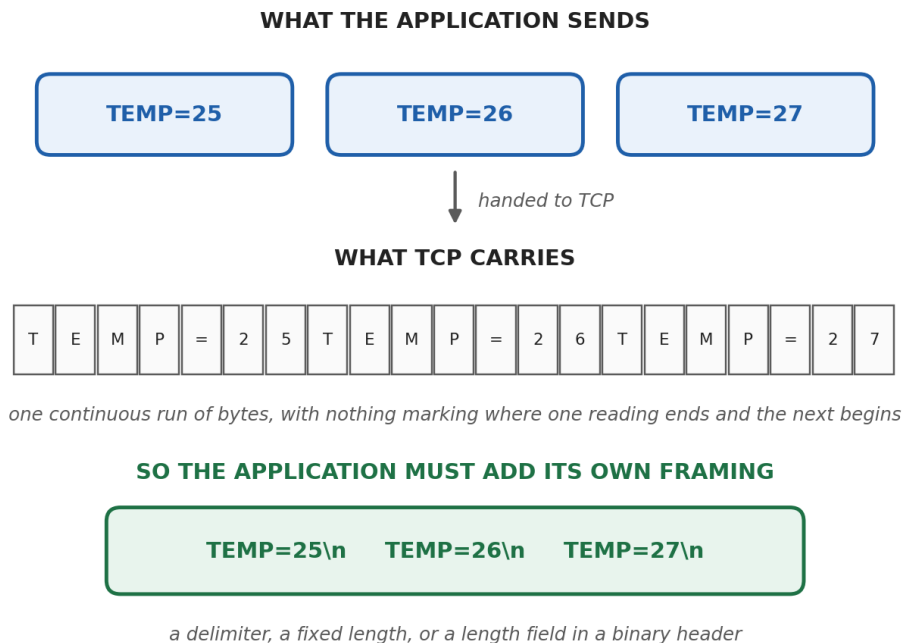


Figure 6. Three readings become one undivided run of bytes. Any structure you need has to be added by the application.

So if the application needs to know where one message ends and the next begins, the application must say so. The common approaches are:

- A delimiter, such as a newline character at the end of every message. Simple, readable, and fine as long as the data itself cannot contain the delimiter.
- A fixed length, where every message is exactly the same number of bytes. Efficient and easy to parse, but inflexible.
- A length prefix, where each message starts with a small header saying how many bytes follow. This is what serious binary protocols do, and it is what MQTT itself does.

This is a large part of the answer to the question of why anyone uses MQTT rather than raw TCP. Framing is one of several problems that MQTT has already solved for you.

1.12 How This Differs from HTTP Request and Response

HTTP is an application layer protocol that runs on TCP, so everything described so far still applies underneath it. What HTTP adds is a very specific conversation shape: the client asks, and the server answers.

```
GET /temperature HTTP/1.1
Host: sensor.local

HTTP/1.1 200 OK
Content-Type: text/plain

25.4
```

That shape has three consequences that matter for embedded work.

1.12.1 The server cannot start a conversation

In the traditional model, a response only exists because a request came first. A server that wants to push a command to a device has no way to do so on its own. The usual workaround is polling, where the device asks every few seconds whether there is anything for it, which wastes bandwidth and still adds delay. Web technologies such as WebSockets and server sent events exist precisely to escape this limitation.

1.12.2 Every request carries its own context

HTTP is stateless by design. Each request has to carry its method, path, host, headers, and often authentication tokens, because the server is not required to remember anything about the last one. That is a sensible trade for the web, where a server may handle millions of unrelated visitors. It is a poor trade for a sensor sending two bytes of data, where the description of the request can be twenty times larger than the data itself.

1.12.3 HTTP does not necessarily open a new connection each time

This is worth stating clearly, because the oversimplified version is repeated everywhere. Since HTTP/1.1, persistent connections are the default: the same TCP connection is reused for multiple requests unless one side closes it. HTTP/2 goes further and multiplexes many requests over a single connection at once.

So the honest distinction is not that HTTP means short connections and raw TCP means long ones. Both can be long lived. The real difference is the conversation model. HTTP always follows ask and answer. Raw TCP lets either side speak at any time.

1.13 How This Differs from MQTT and Its Broker

MQTT is also an application layer protocol running on TCP, but it changes something more fundamental than the message shape. It changes who is connected to whom.

With raw TCP, the ESP32 connects directly to the machine it wants to talk to. With MQTT, every client connects only to a broker. Clients never connect to each other. A device that wants to send information publishes it to a topic, and any device interested in that topic subscribes to it. The broker does the matching and the delivery.

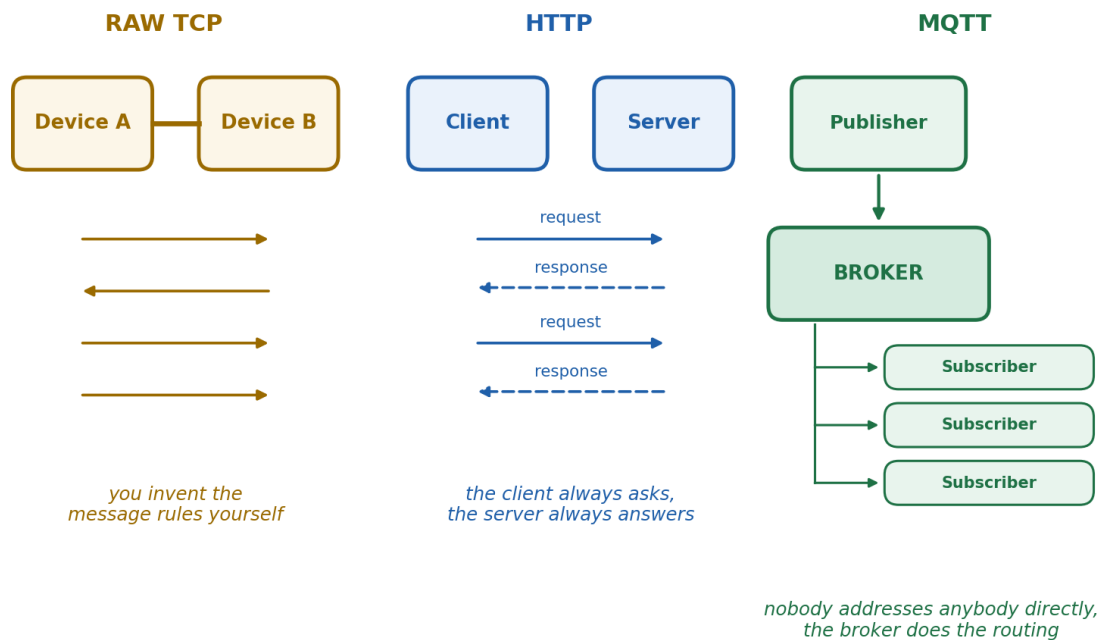


Figure 7. The same three devices under three models. The broker in the third panel is the structural difference, not a performance trick.

This indirection buys a set of properties that are genuinely hard to build by hand:

- Decoupling in space. The publisher does not know the address, or even the existence, of its subscribers.
- Decoupling in number. One publish can reach one subscriber, ten subscribers, or none at all, with no change to the publisher.
- Decoupling in time. With retained messages, a subscriber that connects later can immediately receive the last known value on a topic.
- Quality of service. MQTT defines delivery guarantees, from fire and forget through to exactly once, without the application implementing them.
- Sessions and last will. The broker can remember a client across reconnections and can announce its unexpected disappearance.
- One connection per device. A device with one broker connection can reach every other device on the system through it.

The cost of all this is the broker itself. It is another piece of software that must run somewhere, be reachable, and be secured. In exchange, adding an eleventh device to a ten device system requires no changes anywhere else, which is not true of a mesh of direct TCP connections.

1.14 The Three Models Side by Side

Feature	Raw TCP	HTTP	MQTT
Layer	Transport	Application	Application
Runs on TCP	It is TCP	Yes	Yes
Conversation model	Whatever you design	Request and response	Publish and subscribe

Feature	Raw TCP	HTTP	MQTT
Middleman	None	None	A broker
Who can speak first	Either side	The client only	Either side, through the broker
Persistent connection	Possible and normal	Default since HTTP/1.1	The normal way it is used
Message boundaries	You define them	Defined by the protocol	Defined by the protocol
Addressing	IP address and port	URL and resource path	Topic name
Delivery guarantees	You implement them	Per request	Built in, three QoS levels
Multiple recipients	You implement it	You implement it	Built in
Overhead per message	Potentially the lowest	Usually the highest	Low
Best suited to	Custom links between two known devices	Web APIs and occasional requests	Frequent messaging between many devices

1.15 How Much Does the Wrapper Actually Cost?

It is worth putting numbers on the overhead claim rather than repeating it as folklore. Consider one temperature reading sent three ways.

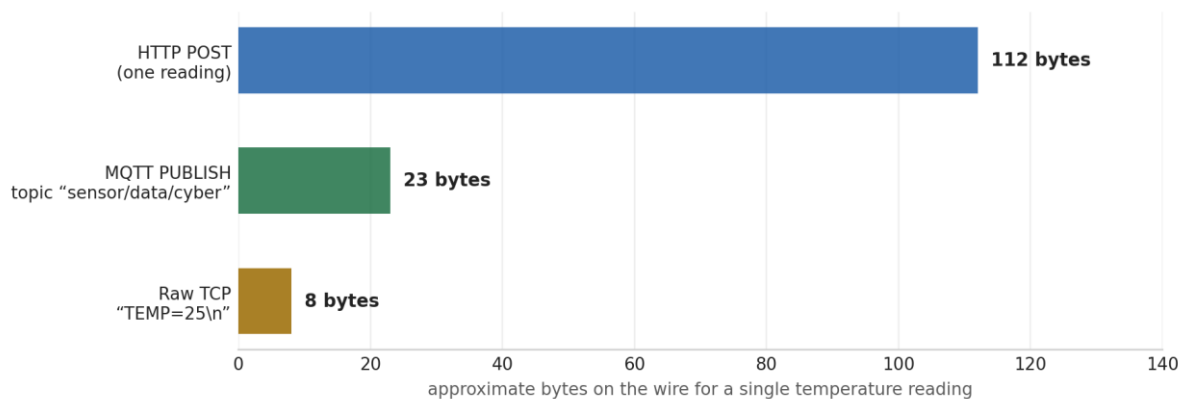


Figure 8. Approximate bytes on the wire for a single reading. The MQTT figure assumes a topic name of seventeen characters, and the HTTP figure counts one modest set of headers. Real values move with topic length and header choices.

Two honest conclusions come out of that chart, and the second one is usually left out.

First, MQTT really is compact compared with HTTP, because it was designed for constrained devices and unreliable links rather than for browsers. Second, raw TCP with a minimal custom protocol can be lighter still, because MQTT has to encode a packet type, a topic, and control information that your own format may not need. MQTT is called lightweight in comparison with HTTP, not in comparison with a hand written protocol.

What MQTT gives you in exchange for those extra bytes is everything listed in Section 1.13. That is usually a very good trade, but it is a trade, not a free win.

1.16 Choosing Between Them

If you need to...	Use	Because
Send frequent readings from many devices to many consumers	MQTT	The broker handles routing, fan out, and delivery guarantees

If you need to...	Use	Because
Expose data to a website or a mobile app	HTTP	Every platform already speaks it, and firewalls expect it
Link two known devices with an unusual data format	Raw TCP	No protocol overhead, and complete control over the format
Receive commands the moment they are issued	MQTT or raw TCP	Both keep a connection open in which the other side can speak first
Send one reading per hour from a battery device	HTTP or MQTT, with reconnect	There is nothing to keep alive between wakeups

1.17 Applying This to the ESP32 Work

It is worth mapping all of this onto the progression the project has already followed.

The earlier ESP-NOW work connected two ESP32 boards directly. There was no broker, no TCP, and no IP addressing in the ordinary sense, because ESP-NOW is a link layer protocol from Espressif that sends short frames straight between devices. It is fast and simple, and it stops at the edge of the local radio range.

Moving to MQTT changed the architecture, not just the library. Each board now opens one TCP connection to a broker, keeps it open, and publishes to topics. The broker decides who receives what. Adding a laptop client such as MQTXX to watch the traffic required no change at all on the ESP32, which is exactly the decoupling described earlier.

If the same system were rebuilt on raw TCP, the ESP32 would open a socket to a specific IP address and port, and every one of the following would become the developer's responsibility: message framing, an addressing scheme, reconnection logic, delivery confirmation, and a way for more than two devices to take part. That list is a fair description of what a broker is for.

In every one of these designs the ESP32 is the client. It is the side that initiates. The broker or the server is the side that waits.

1.18 Section 1 Recap

- TCP provides a reliable, ordered, two-way byte stream between exactly two endpoints.
- The server binds to an address and port and waits. The client initiates. That is the whole distinction.
- The three-way handshake costs one round trip before any application data can move.
- A connection is a channel, not a message, and it stays open until one side closes it.
- Reconnecting per message repeats the transport, security, and application setup every time, which is why frequent traffic belongs on a persistent connection.
- TCP is full duplex, so once the connection exists either side can speak first.
- TCP carries bytes, not messages, so the application must define its own framing.
- HTTP adds a request and response shape on top of TCP, and since HTTP/1.1 it reuses connections by default.
- MQTT adds a broker, topics, and delivery guarantees, which decouples publishers from subscribers entirely.
- MQTT is lightweight compared with HTTP, and heavier than a minimal custom protocol over raw TCP.

GOLDEN RULE

Before choosing a protocol, ask two questions. How often do these devices talk, and who needs to be able to speak first? Frequent traffic argues for a connection that stays open. A need for the server to initiate rules out plain request and response.

Section 2: TLS, What It Actually Adds

Everything in Section 1 assumed a network where nothing goes wrong on purpose. TCP will faithfully deliver a command to stop a motor, and it will just as faithfully deliver a forged one. It has no way to tell the difference, because checking who sent the bytes was never part of its job.

TLS, or Transport Layer Security, is the layer that fills that gap. It is the same technology behind the padlock in a browser, and it is what turns MQTT into MQTTS and HTTP into HTTPS. The current specification is RFC 9846, published in 2026, which replaced RFC 8446 while keeping the protocol itself as TLS 1.3.

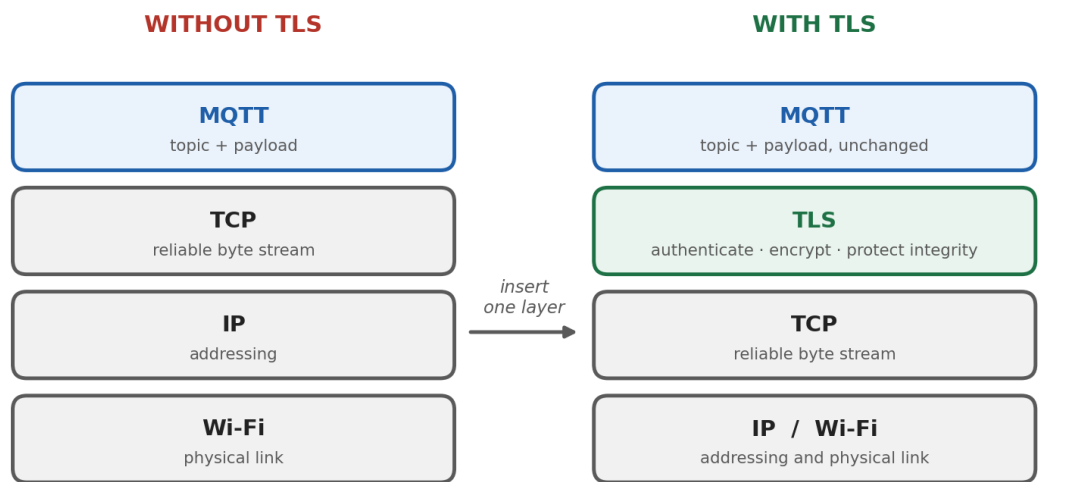
This section explains what TLS adds, what it deliberately leaves alone, and why a device with an unset clock will fail to connect for reasons that look nothing like a clock problem.

THE SINGLE SENTENCE TO REMEMBER

TLS does not change what you say. It changes who can read it, who can prove they are listening, and whether anyone can alter it on the way.

2.1 Where TLS Fits

TLS is inserted between the application protocol and TCP. Nothing above it changes and nothing below it changes. The application still hands down the same bytes, and TCP still carries a stream.



anyone on the path can read "motor/control = ON"

the same MQTT message, now inside an encrypted record

Figure 9. One layer is inserted. The MQTT message above it and the TCP stream below it are untouched.

This is why the same security layer serves so many protocols. HTTPS is HTTP plus TLS plus TCP. MQTTS is MQTT plus TLS plus TCP. A custom protocol can be secured the same way, without redesigning it.

2.2 What Plain TCP Leaves Exposed

Imagine the ESP32 publishing to the topic motor/control with the payload ON, over plain TCP on port 1883. Three separate things can go wrong, and they are worth separating because TLS answers each one differently.

2.2.1 Anyone on the path can read it

The data travels as it was written. On a shared Wi-Fi network, through a router, or across any hop in between, the topic and payload are plainly visible to anything able to observe the traffic. For a temperature reading that may not matter much. For a door lock, a camera feed, or a password sent to a broker, it matters a great deal.

2.2.2 Anyone can pretend to be the broker

The ESP32 connects to an IP address. It has no way of confirming that the machine answering is the broker it intended to reach. An attacker able to influence DNS, or to sit on the same network, can answer instead, collect everything the device publishes, and send back commands of their own. The device would never notice, because nothing in TCP is designed to notice.

2.2.3 Anyone in the middle can change it

TCP checksums catch accidental corruption from noisy links. They are not a defence against deliberate modification, because an attacker who changes the data can simply recalculate the checksum. A payload of OFF can become ON in transit, and the receiving side has no way to tell.

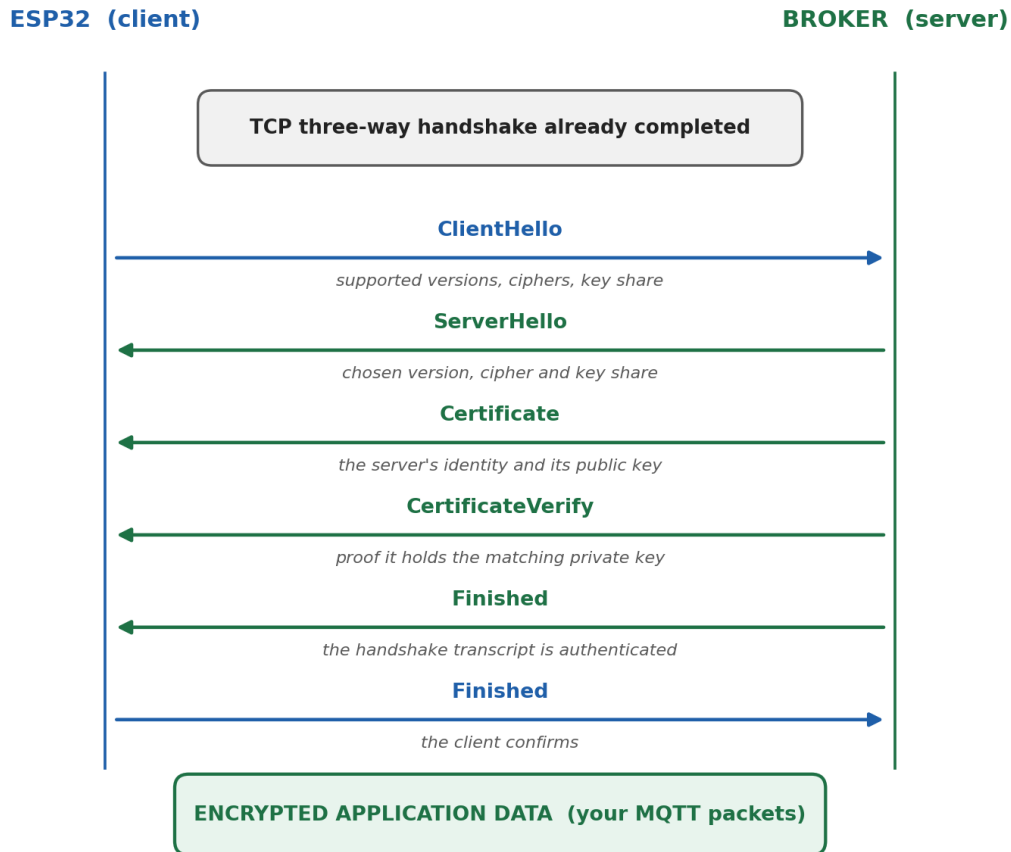
2.3 The Three Guarantees TLS Provides

Property	The question it answers	How TLS provides it
Confidentiality	Can anyone else read this?	The application data is encrypted before it is handed to TCP
Authentication	Am I really talking to the intended server?	The server presents a certificate and proves it holds the matching private key
Integrity	Has anything been altered on the way?	Every protected record carries a cryptographic authentication tag, so tampering is detected

Encryption without authentication would be close to useless in practice. A perfectly encrypted connection to an attacker's server is still a connection to an attacker's server. This is why the certificate check, and therefore the clock, is not an optional refinement.

2.4 The TLS Handshake

Before any protected data moves, TLS runs its own negotiation inside the TCP connection that has already been established. The TLS 1.3 handshake exists to agree the cryptographic parameters, authenticate the peers, and establish shared secret keying material. After that, the agreed keys protect all application traffic.



Simplified. TLS 1.3 allows further optional paths, but the shape is always the same: agree, authenticate, then protect.

Figure 10. A simplified TLS 1.3 handshake. The TCP connection already exists before the first message here is sent.

2.4.1 ClientHello

The device announces which TLS versions it supports, which cryptographic algorithms it can use, and its contribution to the key exchange. Sending the key material in the very first message is what allows TLS 1.3 to complete in a single round trip, where earlier versions needed two.

2.4.2 ServerHello

The server picks one version and one cipher suite from what the client offered and sends its own key exchange contribution. From this point onward, both sides can derive shared keys, and the remainder of the handshake is already encrypted.

2.4.3 Certificate

The server sends its certificate, and usually the intermediate certificates needed to link it back to a trusted root. This is the identity claim.

2.4.4 CertificateVerify

The server signs the handshake transcript with the private key that matches the public key in its certificate. This is the proof behind the claim. Anyone can copy a public certificate, which is why possession of the private key is what actually authenticates the server.

2.4.5 Finished

Both sides send a Finished message that authenticates everything exchanged so far, which stops an attacker from quietly editing the negotiation itself. After this, application data flows inside encrypted records.

2.5 Two Handshakes, Not One

It is easy to blur these together, and separating them makes debugging much easier.

	TCP handshake	TLS handshake
Purpose	Establish a reliable transport connection	Establish a secure channel inside it
Messages	SYN, SYN and ACK, ACK	ClientHello, ServerHello, Certificate, CertificateVerify, Finished
Runs first	Yes	No, it needs TCP to already exist
Involves certificates	No	Yes
Depends on the device clock	No	Yes, for certificate validity
Typical failure message	Connection refused, or timeout	Certificate not yet valid, expired, or unknown CA

If the failure is connection refused, the problem is the address, the port, or a firewall. If TCP connects and the failure appears immediately afterwards, the problem is almost always the certificate, and very often the clock.

2.6 What a Certificate Actually Is

A TLS certificate is a signed statement that binds an identity to a public key. It is not a secret. It is published openly, and anybody can download it.

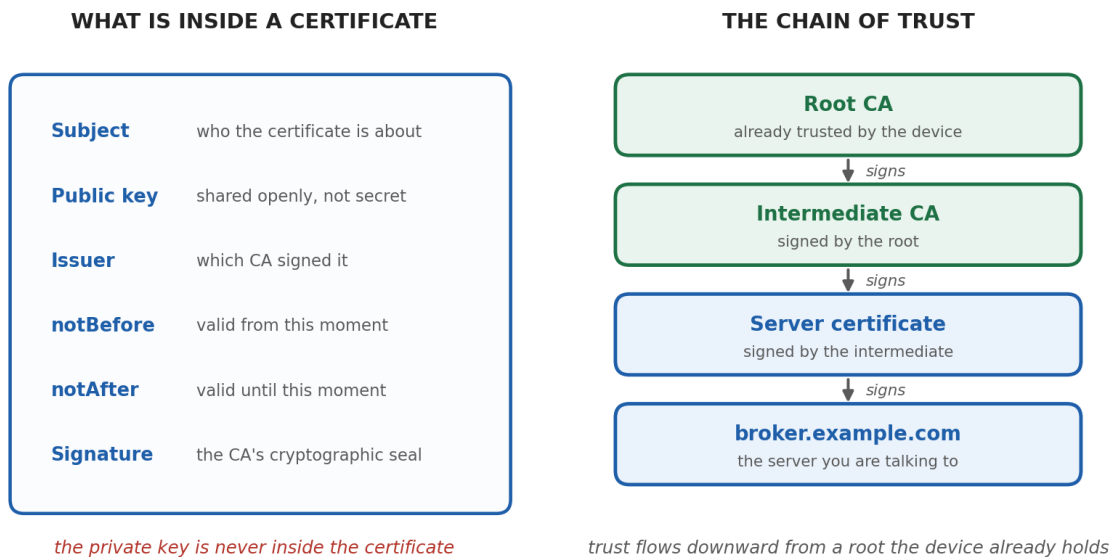


Figure 11. The contents of a certificate, and how one certificate is trusted because of another. The private key never appears in the certificate.

The certificate is signed by a certificate authority, which is an organisation the device already trusts. Usually the server certificate is not signed by a root directly, but by an intermediate that the root signed, forming a chain. The device stores a small set of trusted root certificates, and validation walks the chain upward until it reaches one of them.

For an internal broker on a private network, a self signed certificate or a private certificate authority is perfectly reasonable, provided the device is given that certificate to trust. The mechanism does not change, only the source of the trust.

2.7 What the Device Checks

Presenting a certificate is not the same as passing validation. A client that is doing its job checks all of the following, and failing any one of them should abort the handshake.

1. Does the signature chain lead to a root certificate this device already trusts?
2. Is every signature in that chain mathematically valid?
3. Does the name in the certificate match the hostname the device intended to reach?
4. Is the current time inside the certificate's validity window?
5. Does the server prove possession of the matching private key, through CertificateVerify?
6. Is the certificate allowed to be used for this purpose, according to its key usage extensions?

Item four is the one that turns a security question into an embedded systems question, and it is the reason for the rest of this section.

2.8 TLS Does Not Change Your Data Format

This is a point beginners often get backwards. Adding TLS does not create a different version of MQTT. The application still builds exactly the same packet: the same topic, the same payload, the same quality of service flags. TLS then encrypts the bytes carrying it, and the receiving side decrypts them and hands the identical packet upward.

Without TLS	With TLS
Topic: sensor/data/cyber	Topic: sensor/data/cyber
Payload: 1	Payload: 1
Port 1883	Port 8883
Broker sees an MQTT PUBLISH	Broker sees the same MQTT PUBLISH

The only things that change in the application are the port, the client library configuration, and the certificate the device is told to trust. The protocol logic is untouched.

2.9 What an Observer Can Still See

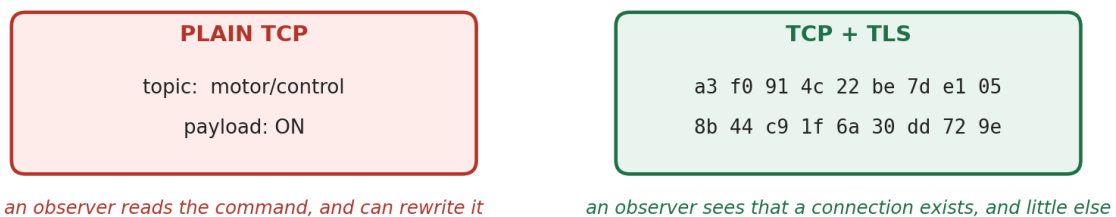


Figure 13. The same publish, observed on the network with and without TLS.

It is worth being precise rather than overselling this. TLS protects the content, not the fact that a conversation is happening. An observer can still see that a device at one IP address is talking to a broker at another, on which port, how often, and roughly how large each message is. In some deployments the server name requested during the handshake is also visible. What the observer cannot do is read the topic, read the payload, or alter either one without detection.

2.10 Why the Clock Matters: notBefore and notAfter

Every X.509 certificate, as defined in RFC 5280, contains two timestamps. notBefore is the moment the certificate becomes valid, and notAfter is the moment it stops being valid. Validation compares the current time against that window.

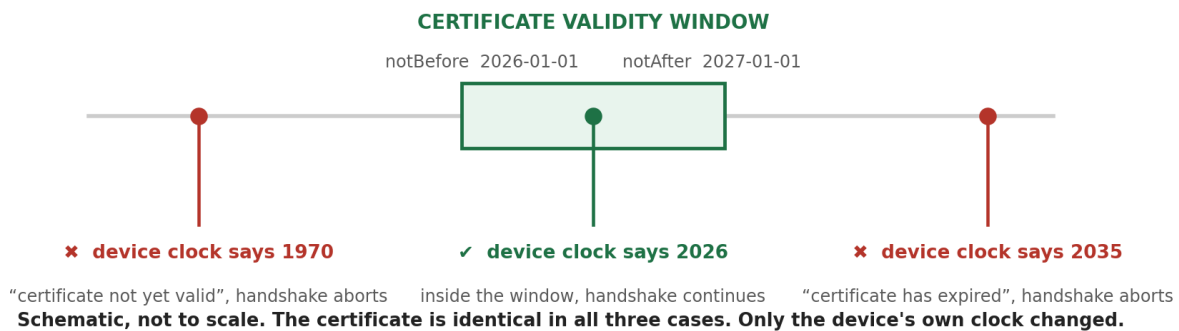


Figure 12. One certificate, three device clocks. The certificate is identical in all three cases. Only the device's own idea of the time has changed.

The reason this check exists is straightforward. Certificates expire so that a key which is stolen, or an identity which is no longer legitimate, does not remain trusted forever. Without a time check, an attacker holding a certificate revoked years ago could keep using it. The validity window is therefore part of the trust decision, not an administrative detail.

But a comparison against the current time requires the device to know the current time. This is where a security requirement becomes a hardware problem.

2.11 The Two Failure Modes

2.11.1 The clock is too early

The ESP32 boots and, with no other source of information, believes the date is 1 January 1970, which is the zero point of Unix time. The certificate says it is valid from 1 January 2026. The device compares the two, concludes that the certificate is not yet valid, and aborts the handshake. The certificate is completely fine. The device is wrong, and the device has no way to know it.

2.11.2 The clock is too late

A device that has drifted badly, or been configured incorrectly, believes it is 2035. The certificate expired in 2027. The device concludes the certificate has expired and aborts. Same outcome, opposite cause.

THE SYMPTOM THAT WASTES THE MOST TIME

Wi-Fi connects. DNS resolves. The broker address and port are correct. The CA certificate is the right one. Everything in the code looks right, and the TLS connection still fails.

Before rereading the certificate handling for the fifth time, print the system time. If it says 1970, the problem is not the certificate.

2.12 Why This Hits Embedded Devices Specifically

A laptop or a phone almost never has this problem, for reasons that are easy to overlook.

- It has a real time clock chip with its own battery, so it keeps time while switched off.
- Its operating system synchronises time in the background as a matter of course.
- By the time any application runs, the clock is already correct.

An ESP32 has none of these guarantees. It does contain an internal real time clock, and that clock keeps counting through deep sleep, which is genuinely useful. What it does not have by default is a backup battery, so a full power cycle leaves it with no idea of the date. On first boot after power up, the system time is simply unset.

millis() IS NOT THE TIME

`millis()` returns the number of milliseconds since the device started running. It is elapsed time, and it is excellent for timing intervals and debouncing.

It cannot tell you that the date is 10 August 2026, and TLS validation needs exactly that. Elapsed time and wall clock time are different concepts, and only one of them can be compared with `notBefore` and `notAfter`.

2.13 NTP and SNTP

The Network Time Protocol is how a device asks the network what time it is. It is one of the oldest protocols still in daily use, it runs over UDP on port 123, and it is designed to work out not just the remote clock's reading but also the delay of the network path, so that the answer can be corrected for travel time.

NTP is organised as a hierarchy. Stratum 0 is a reference source such as an atomic clock or a GPS receiver. Stratum 1 servers are directly attached to those sources, stratum 2 servers synchronise from stratum 1, and so on. Public pools such as `pool.ntp.org` distribute requests across many volunteer servers, which is what most devices actually use.

SNTP, the Simple Network Time Protocol, is a reduced version of the same thing. It uses the identical packet format and the identical servers, and it skips the sophisticated filtering and statistical smoothing that full NTP performs. Since RFC 5905, SNTP is defined as a subset of NTP rather than as a separate protocol. For a microcontroller that needs to know the date to within a second or two, SNTP is entirely sufficient, and it is what ESP-IDF provides.

	NTP	SNTP
Packet format	Identical	Identical
Servers used	The same	The same
Algorithms	Filtering, selection, clustering, discipline	A simple correction from one response
Typical accuracy	Milliseconds or better	Well within a second, which is ample here
Suits	Servers and infrastructure	Microcontrollers and simple clients

2.14 SNTP on the ESP32 in Practice

The API differs slightly between Arduino-ESP32 and ESP-IDF, but the shape is the same in both.

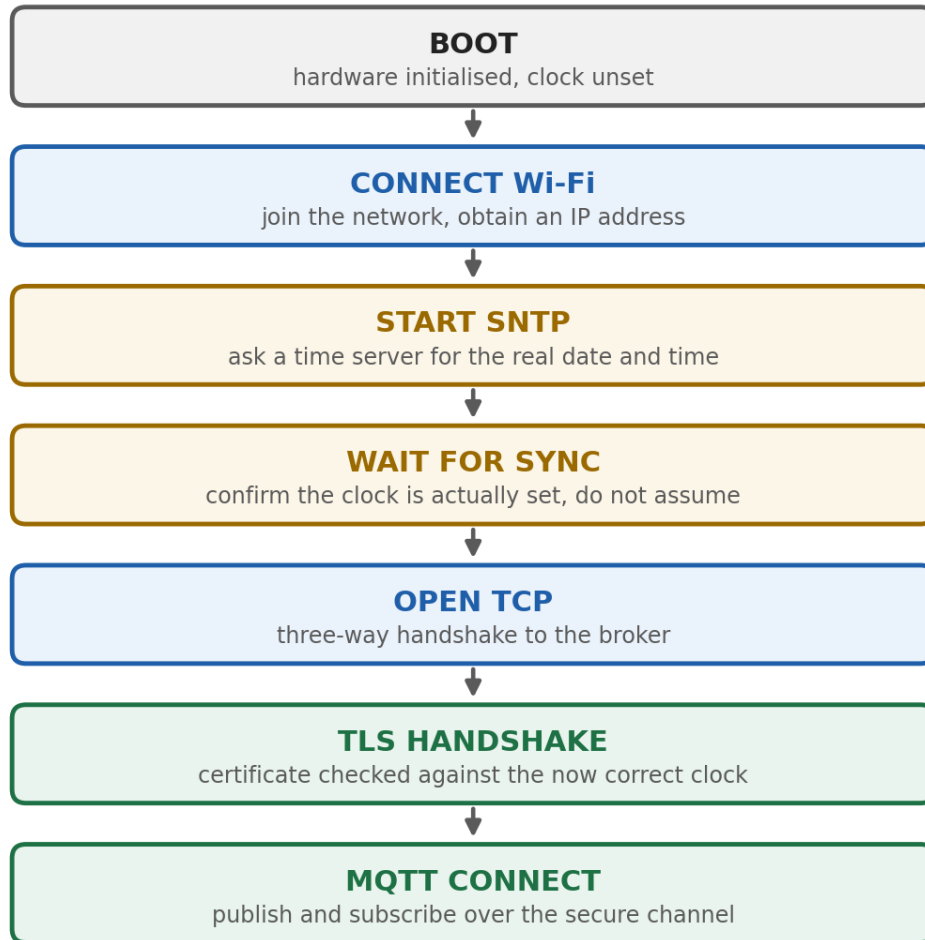
In ESP-IDF, the SNTP service is configured and initialised with `esp_netif_sntp_init`, started with `esp_netif_sntp_start` when it was not auto started at initialisation, and waited on with `esp_netif_sntp_sync_wait`, which returns once the system time has actually been set or times out if it has not. In Arduino, `configTime` performs the equivalent setup, after which the code should poll the clock until it returns a plausible date.

Four practical points are worth more than the function names.

- Wait, do not assume. Starting SNTP and moving straight to TLS is a race the device will sometimes lose. Confirm that the clock has been set before continuing.
- Have a plan for failure. If synchronisation times out because the network is slow or the time server is unreachable, retry with a limit rather than proceeding with an unset clock and blaming the certificate.
- Resynchronise periodically. Cheap oscillators drift, and a device that runs for months will wander. The SNTP service can update the system time at an interval you configure.
- Certificates are checked in UTC. Time zones and daylight saving affect how you display time to a human, not how validity is evaluated. Keep the system clock on UTC and convert only for presentation.

2.15 The Correct Startup Order

Putting the dependencies together gives a startup sequence where each step exists because the step after it requires it.



Each step depends on the one above it. Skip the clock, and the certificate check has nothing to compare against.

Figure 14. The startup dependency chain. Time synchronisation is not an optional extra here, it is a prerequisite for the certificate check.

Reading the chain backwards makes the logic clear. A secure MQTT session needs a TLS handshake. The TLS handshake needs certificate validation. Certificate validation needs the current date. The current date comes from SNTP. SNTP needs a network connection. Therefore Wi-Fi comes first and TLS comes last, and no step in between can be skipped.

2.16 The Whole Picture, End to End

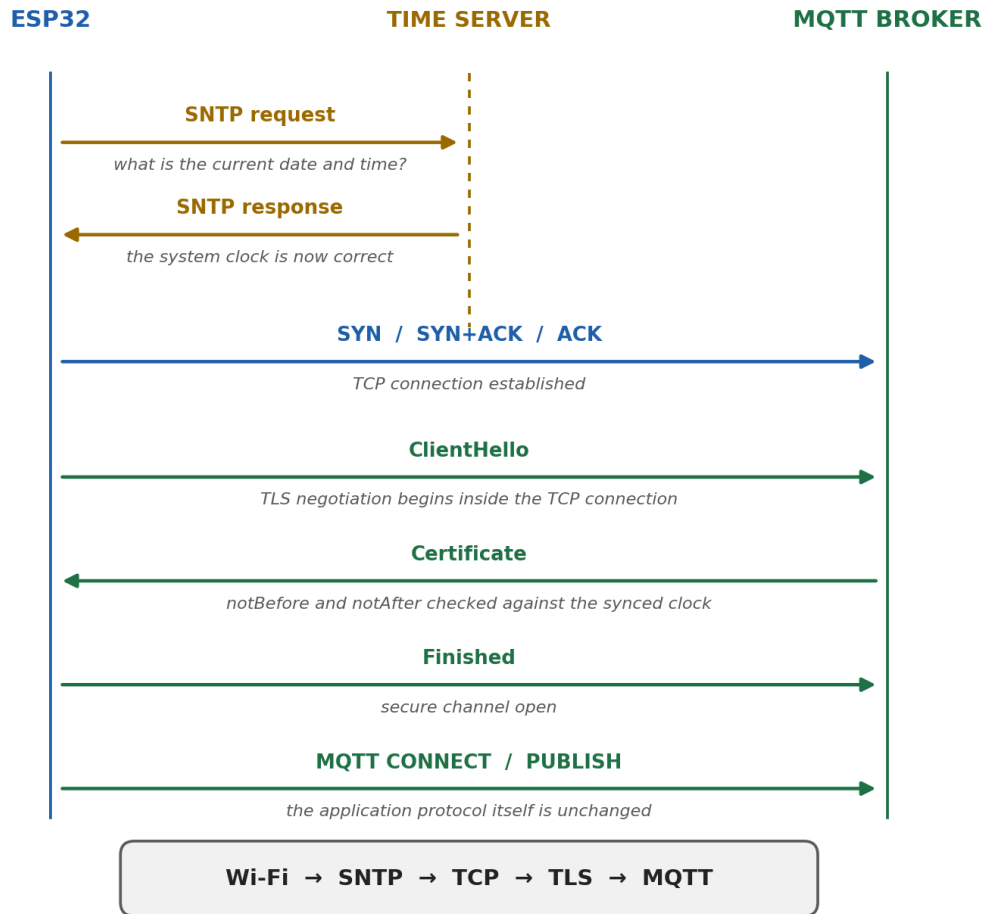


Figure 15. One complete startup, from an unset clock to a published message on a secured connection.

That single diagram contains the whole of both sections. TCP provides a reliable connection. TLS turns that connection into a secure channel. MQTT uses the secure channel to exchange application messages. SNTP, which is not part of any of them, quietly makes the middle step possible.

2.17 A Debugging Checklist

Symptom	Most likely cause	What to check
Connection refused immediately	Nothing is listening	Broker address, port, whether the broker is running, firewall rules
TCP connects, then the handshake fails at once	Certificate validation	The device clock first, then the CA certificate
Error mentions a certificate that is not yet valid	The clock is behind, very often at 1970	Whether SNTP ran and whether the code waited for it
Error mentions an expired certificate	Either a genuinely old certificate or a clock that is ahead	Print the system time and the certificate dates side by side
Unknown or untrusted certificate authority	The device does not hold the right root	The CA certificate compiled into the firmware
Hostname mismatch	Connecting by IP address to a certificate issued for a name	Use the hostname the certificate was issued for

Symptom	Most likely cause	What to check
Works on the bench, fails after a power cut	The clock was set once and lost on power up	Synchronise at every boot, not only at first flash
Works for weeks, then fails	Drift, or a certificate that reached its expiry	Periodic resynchronisation and certificate renewal

2.18 Common Misconceptions

The claim	Why it is wrong
TLS replaces TCP	TLS runs on top of TCP and depends on it for reliable delivery
TLS turns MQTT into a different protocol	The MQTT packets are identical. Only the channel carrying them changed
MQTT is faster than TCP	MQTT runs on TCP, so it cannot be faster than its own transport. The meaningful comparison is MQTT against HTTP
SNTP is part of TLS	They are unrelated. SNTP sets the clock, and TLS reads it during validation
A Wi-Fi connection means the device knows the time	Wi-Fi provides connectivity. The clock is still unset until something synchronises it
millis() gives the current time	It gives elapsed time since boot, which cannot be compared with a certificate date
Encryption alone is enough	Encrypting a connection to an impostor protects the impostor. Authentication is the half that matters here
HTTP always opens a new TCP connection per request	HTTP/1.1 reuses connections by default, and HTTP/2 multiplexes many requests over one

2.19 Section 2 Recap

- TLS sits between the application protocol and TCP, and leaves both untouched.
- It provides confidentiality, authentication, and integrity. Encryption alone would not be enough.
- The TLS handshake runs inside an existing TCP connection and is a separate step from the TCP handshake.
- A certificate binds an identity to a public key and is signed by an authority the device already trusts.
- Validation checks the chain, the signatures, the hostname, the key possession, and the validity dates.
- The validity dates are `notBefore` and `notAfter`, so validation needs the real date and time.
- An ESP32 boots without a reliable wall clock, and `millis()` is elapsed time, not a date.
- SNTP obtains the real time over the network, and the code must wait for it and confirm it worked.
- The dependency chain is Wi-Fi, then SNTP, then TCP, then TLS, then MQTT.
- A TLS failure that appears immediately after a successful TCP connection is a clock problem until proven otherwise.

GOLDEN RULE

TLS asks one question at the start of every connection: is this certificate valid right now? A device that cannot answer the second half of that question will fail the first half, no matter how correct the rest of the code is.

Final Takeaway

Three layers, three different questions, and each one only answers its own.

Layer	The question it answers
TCP	How can two endpoints exchange bytes reliably and in order?
TLS	Am I talking to the right party, and can anyone read or change what we say?
MQTT	How do many devices exchange messages through a broker without knowing about each other?

TCP gives you a reliable connection. TLS turns that connection into a secure channel. MQTT uses the secure channel to move application messages between devices that never address each other directly. Underneath all three sits one embedded systems detail that belongs to none of them: a device that does not know what day it is cannot verify anything, and will fail with an error that points at everything except the real cause.

References

Primary specifications are cited in their current form. Where a widely quoted document has since been replaced, the superseding version is given, with the older number noted for continuity.

Transport and application protocols

- Eddy, W. (Ed.). Transmission Control Protocol (TCP). RFC 9293, IETF, August 2022. Obsoletes RFC 793. <https://www.rfc-editor.org/info/rfc9293>
- Fielding, R., Nottingham, M., Reschke, J. (Eds.). HTTP Semantics. RFC 9110, IETF, June 2022. <https://www.rfc-editor.org/info/rfc9110>
- Fielding, R., Nottingham, M., Reschke, J. (Eds.). HTTP/1.1. RFC 9112, IETF, June 2022. Persistent connections are the default. <https://www.rfc-editor.org/info/rfc9112>
- OASIS. MQTT Version 5.0, OASIS Standard, March 2019. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- GeeksforGeeks. What is Transmission Control Protocol (TCP). <https://www.geeksforgeeks.org/computer-networks/what-is-transmission-control-protocol-tcp/>
- Khan Academy. Transmission Control Protocol (TCP). <https://www.khanacademy.org/computing/computers-and-internet/xcae6f4a7ff015e7d:the-internet/xcae6f4a7ff015e7d:transporting-packets/a/transmission-control-protocol--tcp>
- TechTarget. TCP (Transmission Control Protocol) definition. <https://www.techtarget.com/searchnetworking/definition/TCP>
- CDEBYTE. What is MQTT and how it relates to TCP. <https://www.cdebyte.com/news/733>

Transport Layer Security and certificates

- Rescorla, E. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 9846, IETF, 2026. Obsoletes RFC 8446, which is the version most existing material cites. <https://www.rfc-editor.org/info/rfc9846>
- Rescorla, E. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446, IETF, August 2018. Superseded, retained here for reference. <https://www.rfc-editor.org/info/rfc8446>
- Cooper, D. et al. Internet X.509 Public Key Infrastructure Certificate and CRL Profile. RFC 5280, IETF, May 2008. Defines notBefore and notAfter. <https://www.rfc-editor.org/info/rfc5280>
- Cloudflare Learning Center. What is Transport Layer Security (TLS)? <https://www.cloudflare.com/learning/ssl/transport-layer-security-tls/>
- Khan Academy. Transport Layer Security protocol (TLS). <https://www.khanacademy.org/computing/computers-and-internet/xcae6f4a7ff015e7d:online-data-security/xcae6f4a7ff015e7d:secure-internet-protocols/a/transport-layer-security-protocol-tls>
- Khan Academy. Digital certificates and public key infrastructure. <https://www.khanacademy.org/computing/computers-and-internet/xcae6f4a7ff015e7d:online-data-security/xcae6f4a7ff015e7d:secure-internet-protocols/a/digital-public-key-certificates>
- GeeksforGeeks. Transport Layer Security (TLS). <https://www.geeksforgeeks.org/computer-networks/transport-layer-security-tls/>

Time synchronisation

- Mills, D. et al. Network Time Protocol Version 4: Protocol and Algorithms Specification. RFC 5905, IETF, June 2010. Defines SNTP as a subset of NTP and obsoletes RFC 4330. <https://www.rfc-editor.org/info/rfc5905>
- TimeTools. SNTP overview. <https://timetoolsltd.com/ntp/sntp-overview/>
- NTP Pool Project. pool.ntp.org. <https://www.ntppool.org/>

ESP32 implementation

- Espressif. System Time, ESP-IDF Programming Guide. SNTP initialisation, sync modes and waiting for synchronisation. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/system_time.html
- Espressif. ESP-NETIF Programmers Manual. esp_netif_sntp_init, esp_netif_sntp_start and esp_netif_sntp_sync_wait. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_netif_programming.html
- The ESP Journal. ESP32, TLS and IoT devices. <https://medium.com/the-esp-journal/esp32-tls-transport-layer-security-and-iot-devices-3ac93511f6d8>

- Anedya. Why the right TLS configuration matters in IoT. <https://medium.com/anedya/why-right-tls-configuration-matters-in-iot-17d81431e40f>
-

All figures in this report were drawn specifically for it. They are original schematic illustrations and are not reproduced from any of the sources listed above.